Final Year Project Documentation

Campus Navigation Guide



By

Zeenat Shafiq 076934

Aqsa Khalid 076959

Areej Sajjad 076961

Project ID BSIT22-G24

Under the Supervision of

Professor Usman Ahmad Awan

Bachelor of Science in Information Technology,

Session (2022-2026).

FACULTY OF COMPUTING & INFORMATION TECHNOLOGY,

GOVT. GRADUATE ISLAMIA COLLEGE, GUJRANWALA

Campus Navigation Guide

A project represented to

University of the Punjab, Lahore.

In partial fulfilment of

the requirement for the degree of

Bachelor of Science in Information Technology,

Session (2022-2026).

By

Zeenat Shafiq 076934

Aqsa Khalid 076959

Areej Sajjad 076961

FACULTY OF COMPUTING & INFORMATION TECHNOLOGY,

GOVT. GRADUATE ISLAMIA COLLEGE, GUJRANWALA

DECLARATION

We hereby declare that this software, neither the whole nor a part, was copied from any source. It is further declared that we have developed this software and accompanying report based on our efforts. If any part of this project is proven to be copied from any source or found to be a reproduction of some other, we will stand by the consequences. No portion of the work presented has been submitted for any application for any other degree or qualification of this or any other university or institute of learning.

Zeenat Shafiq [076934]

Signature: _____

Aqsa Khalid [076959]

Signature: _____

Areej Sajjad [076961]

Signature: _____

CERTIFICATE OF APPROVAL

It is to certify that the final year project (FYP) of BSIT, session (2022-26), the project Campus Navigation Guide: A Web-App was developed by Zeenat Shafiq (076934), Aqsa Khalid (076959), and Areej Sajjad (076961) under the supervision of Prof. Muhammad Usman Ahmad Awan, in my opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in Information Technology.

FYP Advisor: Prof. Muhammad Usman Ahmad Awan

Advisor Signature: _____

Signatures (Faculty Advisory Committee (FAC))

	FAC1	**FAC2**
Name		
Signature		

Head of FYP Coordination Office: Prof. Muhammad Usman Ahmad Awan

Signature: _____

Head of Department, Information Technology: Prof. Muhammad Usman Ahmad Awan

Signature: _____ Dated: _____

Executive Summary

The Campus Guide is a web-based application developed to make campus navigation and support easier for students, visitors, and faculty. The system provides an interactive map where users can search any campus location, view building details, and get a simple route to reach their destination. It also shows the user's current position when allowed. The help desk module lets users submit questions or issues directly through the system, reducing the need to visit offices for basic information.

The Software Design Specification describes how the system is structured and how each part works together. It explains the overall architecture, database design, interface layout, data flow, and main components used in the development. The document outlines the design decisions and provides models that support the systems features. The goal of this SDS is to ensure that the design is clear, consistent and matches the functional requirement defined in the project.

Overall, this system is designed to improve the overall campus experience by making navigation more efficient and less time-consuming. Instead of asking others for directions or searching manually for places, users can easily rely on the interactive map to find accurate locations within seconds. The inclusion of real-time location (when enabled) further enhances usability by helping users understand exactly where they are on campus and how they can reach their destination in the shortest and simplest way.

Similarly, the help desk feature plays an important role in improving communication between users and the campus administration. It provides a centralized platform where students and visitors can send their queries without physically going to different departments. This not only saves time but also helps in managing and tracking requests more effectively.

In short, the Campus Guide system combines navigation and support services in one platform, making it a practical and user-friendly solution for improving campus accessibility and communication.

Acknowledgment

We express our gratitude to our Almighty Allah, and after that, we are thankful to our families, who allowed us to get higher education, and we became capable of creating something productive. We're also thankful to our project advisor and department faculty for their support.

Zeenat Shafiq [076934]

Signature: _____

Aqsa Khalid [076959]

Signature: _____

Areej Sajjad [076961]

Signature: _____

Table of Content

Contents

1. Introduction 2

1.1 Problem Statement 2

1.2 Problem Solution 2

1.3 Objectives of the proposed system 2

1.4 Scope 3

1.5 System Components 3

1.5.1 Module 1: Module Name 4

1.6 Related System Analysis / Literature Review 5

1.7 Vision Statement 6

1.8 Tools and Technologies 6

1.9 Project Deliverables 7

1.10 Project Planning 8

2. Analysis 11

2.1 User Classes and Characteristics 11

2.2 Requirements Identifying Techniques 11

2.3 Functional Requirements 12

2.3.1 Functional Requirement 1 12

2.3.2 Functional Requirement 2 13

2.3.3 Functional Requirement 3 13

2.3.4 Functional Requirement 4 14

2.3.5 Functional Requirement 5 14

2.3.6 Functional Requirement 6 15

2.4 Non-Functional Requirements 15

2.5.1 Use case 1 16

2.5.2 Use case 2 18

2.5.3 Use case 3 19

2.5.4 Use case 4 20

2.5.5 Use case 5 21

2.5.6 Use case 6 23

2.5.7 Use case 7 24
2.6 Use Case Diagram 1 26
2.7 Use Case Diagram 2 26
3. System Design 29
3.1 Design Consideration 29
3.2 Requirements Traceability Matrix 30
3.3 Design Models 32
3.3.1 Class Diagram 32
3.3.2 Sequence Diagram 33
3.3.3 State Transition Diagram 37
3.5 Data Design 41
3.6 Data Dictionary 42
3.7 User Interface Design 43
3.8 Design Decisions 45
4. Implementation 48
4.1 Algorithms 48
4.2 External APIs/SDKs 53
4.3 Code repository 54
4.3.1 Matrices of the git repository 54
5. Testing and Evaluation 56
5.1 Unit Testing 56
5.1.1 Unit Test Case 1: User Login 56
5.1.2 Unit Test Case 2: Search Location 57
5.1.3 Unit Test Case 3: Submit Help Desk Query 57
5.1.4 Unit Test Case 4: Admin Reply 58
5.2 Functional Testing 59
5.2.1 Functional Test Case 1: Search Location 59
5.2.2 Functional Test Case 2: Map Display 59
5.2.3 Functional Test Case 3: Route Generation 60
5.2.4 Functional Test Case 4: Help Desk System 61
5.3 Integration Testing 61
5.3.1 Integration Test Case 1: Login + Dashboard 61

5.3.2 Integration Test Case 2: Search + Map Display 62
5.3.3 Integration Test Case 3: Search + Route Navigation 63
5.3.4 Integration Test Case 4: Help Desk + Database 63
5.3.5 Integration Test Case 5: Admin + Help Desk 64
5.4 Performance Testing 65
5.4.1 Performance Test Case 1: Map Load Time 65
5.4.2 Performance Test Case 2: Route Generation Time 66
5.4.3 Performance Test Case 3: Database Response Time 66
5.4.4 Performance Test Case 4: Help Desk Response Time 67
6. System Conversion 70
6.1 Conversion Method 70
6.2 Deployment 70
6.2.1 Deployment Steps 70
6.2.2 Data Conversion 71
6.2.3 Training 71
6.3 Post Deployment Testing 73
6.3.1 Post Deployment Testing Steps 73
6.4 Challenges and Solutions 73
7. Introduction 77
7.2 Evaluation 77
7.2 Traceability matrix 78
7.3 Conclusion 78
7.4 Future work 79

List of Tables

Table 1.1: Tools and Technologies 6
Table 2.1: User and Characteristics 11
Table 2.2: Requirements Identifying Techniques 11
Table 2.3: Functional Requirements 1 12
Table 2.4: Functional Requirement 2 13
Table 2.5: Functional Requirement 3 13
Table 2.6: Functional Requirement 4 14
Table 2.7: Functional Requirement 5 14
Table 2.8: Functional Requirement 6 15
Table 2.9: Non-Functional Requirements 15
Table 2.10: Use case 1 16
Table 2.11: Use case 2 18
Table 2.12: Use case 3 19
Table 2.13: Use case 4 20
Table 2.14: Use case 5 21
Table 2.15: Use case 6 23
Table 2.16: Use case 7 24
Table 3.1: Requirement Traceability Matrix 30
Table 3.2: Data Dictionary 42
Table 4.1: Algorithm 1 - View map 48
Table 4.2: Algorithm 2 – Search location 48
Table 4.3: Algorithm 3 – Route guidance 49
Table 4.4: Algorithm 4 – Submit query 50
Table 4.5: Algorithm 5 – Admin login 50
Table 4.6: Algorithm 6 – View queries 51
Table 4.7: Algorithm 7 – Reply to queries 52
Table 4.8: External APIs/SDKs 53
Table 5.1: Unit Test Case 1: User Login 56
Table 5.2: Unit Test Case 2: Search Location 57
Table 5.3: Unit Test Case 3: Submit Help Desk Query 57
Table 5.4: Unit Test Case 4: Admin Reply 58
Table 5.5: Functional Test Case 1: Search Location 59
Table 5.6: Functional Test Case 2: Map Display 59
Table 5.7: Functional Test Case 3: Route Generation 60
Table 5.8: Functional Test Case 4: Help Desk System 61
Table 5.9: Integration Test Case 1: Login + Dashboard 61
Table 5.10: Integration Test Case 2: Search + Map Display 62
Table 5.11: Integration Test Case 3: Search + Route Navigation 63
Table 5.12: Integration Test Case 4: Help Desk + Database 63
Table 5.13: Integration Test Case 5: Admin + Help Desk 64

Table 5.14: Performance Testing Results Table 65
Table 5.15: Performance Test Case 1: Map Load Time 65
Table 5.16: Performance Test Case 2: Route Generation Time 66
Table 5.17: Performance Test Case 3: Database Response Time 66
Table 5.18: Performance Test Case 4: Help Desk Response Time 67
Table 6.1: Deployment Steps 70
Table 6.2: Data Conversion 71
Table 6.3: Training Instruction 1: Navigation 71
Table 6.4: Training Instruction 2: Route Guidance 72
Table 6.5: Training Instruction 3: Help Desk 72
Table 6.6: Post Deployment Testing Steps 73
Table 6.7: Challenges and Solutions 74
Table 7.1: Evaluation 77
Table 7.2: Traceability matrix 78

List of Figure

Figure 1.1: Gantt chart 8
Figure 2.1: Use Case Diagram 1 26
Figure 2.2: Use Case Diagram 2 26
Figure 3.1: Class Diagram 32
Figure 3.2: Campus Navigation Module 33
Figure 3.3: User Account Module 34
Figure 3.4: Admin Module 35
Figure 3.5: Help Desk Module 36
Figure 3.6: State Transition Diagram 37
Figure 3.7: UML Class Diagram 38
Figure 3.8: UML Component Diagram 39
Figure 3.9: ER Diagram 40

Chapter 1

Introduction

1. Introduction

University campuses are usually very large so students and visitors often find it difficult to locate departments, offices, and other places. They usually ask others for directions which takes time and create confusion. Also for small queries students have to visit help desk physically. To solve these problems our project is developed. It is a web-based system that helps user to find locations easily using digital campus map. The system also provides an online help desk where users can submit their queries. Admin can view and reply these queries. This system saves time and makes campus navigation simple and easy.

1.1 Problem Statement

Students and visitors don't have a central place where they can see the campus location and get basic help. This creates confusion and time waste. There is a need for a simple web-based system that shows a campus map and provides a help desk for common issues.

1.2 Problem Solution

The solution to this problem is a web-based campus navigation guide system. It provides a single platform where users can view the campus map, search locations and get route guidance easily. It also includes an online help desk system where users can submit their queries. This system reduces confusion, saves time, and makes everything easier

1.3 Objectives of the proposed system

- Show campus map with buildings, department markers and admin manage campus location
- Allow users to search locations on the campus
- Display the user's current location using browser GPS
- Provide simple route guidance from user location to the selected place
- Displays department details including timing, contact information, and description
- Let users submit help desk queries and can track the status of submitted queries
- Allow admin to read and reply to help desk queries
- Store all queries and responses in the database for record keeping
- Provide a clean and simple interface for easy navigation and help access
- Reduce the time students spend finding rooms, departments or offices

1.4 Scope

The scope of this project is to develop a web-based system that helps users find locations inside the campus and get basic help. It includes a digital map for searching places and showing routes, and a help desk where users can submit queries. The system is limited to one campus and does not include mobile app, AI features or indoor navigation.

1.5 System Components

We have following system components are:

- Map Navigation module
- Help desk module
- Admin panel
- User module
- Frontend
- Backend
- Database

Modules/System components of the project

1. **Map navigation module**

- Show interactive campus map
- Display markers for buildings and departments
- Detect user location using GPS
- Provide route from current location to destination

2. **Help desk module**

- Submit queries through form
- Send complaints or information requests
- Store queries in database
- Reduce need for physical visit

3. **Admin module**

- View user queries
- Reply to queries
- Manage records

4. **User module**

- Search locations
- View routes
- Submit help desk queries

5. **Frontend**

- Display campus map
- Search locations (departments, offices etc.)
- Show route guidance
- Help desk form for submitting queries
- User friendly interface for easy navigation

6. **Backend**

- Process helpdesk form submissions
- Manage user requests
- Handle admin responses
- Connect frontend with database

7. **Database**

- User queries
- Admin replies
- Campus locations

1.5.1 Module 1: Module Name

Here is the list of features associated with each component of the system:

1. **Client Web App (User Panel)**

Features:

- User registration and login
- Search campus locations
- Select source and destination
- View route on map using Leaflet
- Submit help desk query
- View replies from admin

2. **Admin Web App (Admin Panel)**

Features:

- Admin login with secure access
- Add, update, and delete locations
- Manage routes between locations
- View all user queries
- Reply to user queries
- Monitor system activities

3. **Backend Server (PHP + MySQL)**

Features:

- Handle user and admin requests
- Store and retrieve data from database
- Manage authentication system
- Store queries and replies
- Provide data to frontend

1.6 Related System Analysis / Literature Review

Here is the analysis of the related systems as follows:

Application Name	Weakness	Proposed Project Solution
Google Maps	Not specific for campus navigation	Provides campus-specific navigation with accurate locations
Generic Map Systems	Do not provide internal campus routes	Shows exact routes inside campus
Manual Guidance	Time consuming and unreliable	Provides quick and automated navigation system

1.7 Vision Statement

Here's a vision statement for Campus Navigation Guide system:

For [Students and Visitors]

Who [need an easy way to find campus locations]

The [Campus Navigation Guide]

Is [a web-based navigation system]

That [provides accurate routes and location search inside campus]

Unlike [manual guidance or general map systems]

Our product [is simple, fast, and user-friendly navigation solution]

System Limitations and Constraints

Limitations:

1. **Internet Dependency:** The system requires internet connection to load maps and data.
2. **Device Compatibility:** The system may not work properly on very old devices or browsers.
3. **Map Accuracy:** Route accuracy depends on the available location data.

Constraints:

1. **Scope:** The system is limited to campus navigation only. It helps users find locations and routes inside campus.
2. **Technology Stack:** The system is developed using PHP, MySQL, HTML, CSS, JavaScript, and Leaflet library.

1.8 Tools and Technologies

Table 1.1: Tools and Technologies

	Tools	Version	Rationale
	Visual Studio Code	1.90+	Easy to use, free code editor for writing HTML, CSS, JS and PHP.

Tools, Libraries and Technologies	XAMPP	8.2.4	Runs PHP8.2 and MySQL 8.2 locally for development.
	PhpMyAdmin	5.2.1	Helps manage the MySQL database through a simple web interface.
	Libraries	**Version**	**Rationale**
	Leaflet.js	1.9.4	Lightweight, free library to display interactive maps using OpenStreetMap.
	Leaflet Routing Machine	3.2.12	Used to show simple route paths between two campus points.
	Bootstrap	5.3	Makes frontend design easier with ready-made responsive.
	Technology	**Version**	**Rationale**
	HTML	HTML5	Standard language for creating webpages.
	CSS	CSS3	Used to style and design all web pages.
	PHP	8.2	Backend language used to process forms and manage the help desk.
	JavaScript	ES6+	Handles map functions, search and interactive features.
	MySQL	8.0	Stores user queries, admin responses, and campus location data.

1.9 Project Deliverables

There is a list of project deliverables for Campus Navigation Guide System:

1. **Planning Document**

- **Purpose:** Outlines the project goals, timelines, milestones and required resources for developing the campus navigation and helpdesk system.
- **Contents:** Project scope, objectives, development schedule, resource allocation and risk management plan.
- **Appendix in FYP Report:** This document will be included in the initial phase of the report and provide an overview of the project planning and approach.

2. **Software Requirement Specification (SRS)**

- **Purpose:** The system defines the functional and non-functional requirements of the campus navigation and helpdesk system.
- **Contents:** Use cases, user stories, system features like map navigation, search helpdesk, system interfaces and performance requirements.
- **Appendix in FYP Report:** This document will provide complete system requirements including both functional and non-functional aspects.

3. **Prototypes**

- **Purpose:** It provides a visual representation of the user interface (UI) and user experience.
- **Contents:** Interactive Static designs of web pages including map interface, search and helpdesk forms.
- **Appendix in FYP Report:** These prototypes will help to visualize the system before development and guide implementation.

1.10 Project Planning

Gantt chart representing the plans of carrying out the project activities representing the milestones as well as resources required for each task:

ID	Task Name	Duration					
		Week 2-4	Week 5-10	Week 11-16	Week 17-28	Week 29-30	Week 31-32
1	Planning						
2	Requirement Gathering and Analysis						
3	Design						
4	Coding						
5	Testing						
6	Implementation						

Figure 1.1: Gantt chart

Summary

Here is a summary of key points for the Campus Navigation Guide System:

Key Points

1. **Objective:** Simplify campus navigation, reduce time spent in finding locations and improve communication between students and administration.
2. **Features:** Campus map display, location search, route guidance, current location detection (GPS) and online helpdesk system for queries and responses.
3. **Benefits:** Easy navigation, time-saving, reduced confusion, quick access to information, improved efficiency and better user experience.
4. **Technologies:** HTML, CSS, JavaScript, PHP, MySQL, Leaflet.js, Leaflet Routing Machine and Bootstrap.

Chapter 2

Requirements Analysis

2. Analysis

The Campus Navigation Guide is developed as a web-based application. It outlines the system objectives to provide an easy location search, route navigation and helpdesk support within a campus along with the scope of improving user convenience and reducing manual guidance.

2.1 User Classes and Characteristics

Table 2.1: User and Characteristics

User Classes	Characteristics
Student	The student uses the system to search campus locations, find routes and submit helpdesk queries. It requires a simple and fast navigation features.
Faculty/Staff	The faculty uses the system to navigate between departments and buildings. It may also use helpdesk services for reporting issues.
Admin	The admin manages the user accounts, handles helpdesk queries and responds to user requests. It requires full system access and control.
Guest User	Limited access user who can view basic location information but cannot use advanced features like helpdesk.

2.2 Requirements Identifying Techniques

Each **use case** focuses on a specific goal or action the user can perform in the system:

Table 2.2: Requirements Identifying Techniques

Use Case Name	Associated Functional Requirement
Register User	The system allows students to create a new account in the system.
Login to the System	The system enables users to log in securely using credentials.
Location Search	The system allows users to search for any building, department or rooms on campus.

Route Navigation	The route navigation provides a shortest path from current location to destination.
Helpdesk Query Submission	The system allows users to submit queries or issues related to navigation.
Helpdesk Response System	The admin can view and respond to user queries and provide solutions.

2.3 Functional Requirements

This section describes the functional requirements of the system expressed in the natural language style. This section is organized by features as a system feature name and specific functional requirements associated with this feature. So, for each module and the feature of functional requirements are itemized below:

2.3.1 Functional Requirement 1

Table 2.3: Functional Requirements 1

Identifier	FR-1
Title	Location Search
Requirement	The user shall be able to search for locations within the campus.
Source	Business Requirement
Rationale	The system helps users quickly to find desired places inside the campus.
Business Rule	The search results must match the available campus locations.
Dependencies	FR-4
Priority	High

2.3.2 Functional Requirement 2

Table 2.4: Functional Requirement 2

Identifier	FR-2
Title	Route Navigation
Requirement	The system shall provide the shortest path between selected source and destination locations within the campus.
Source	Business Requirement
Rationale	Improves user experience by guiding users efficiently.
Business Rule	The system must calculate the shortest route.
Dependencies	FR-1
Priority	High

2.3.3 Functional Requirement 3

Table 2.5: Functional Requirement 3

Identifier	FR-3
Title	User Registration
Requirement	The user shall be able to create an account by providing name, email and password.
Source	Business Requirement
Rationale	The system allows new users to access system features.
Business Rule	The email must be unique for each user.
Dependencies	None
Priority	High

2.3.4 Functional Requirement 4

Table 2.6: Functional Requirement 4

Identifier	FR-4
Title	User Login
Requirement	The user shall be able to log in using registered email and password.
Source	Business Requirement
Rationale	Ensures secure access to the system**.**
Business Rule	The user must enter the valid credentials to log in.
Dependencies	FR-3
Priority	High

2.3.5 Functional Requirement 5

Table 2.7: Functional Requirement 5

Identifier	FR-5
Title	Helpdesk Query Submission
Requirement	The user shall be able to submit queries or issues through the helpdesk module.
Source	Business Requirement
Rationale	The system provides a way for users to submit problems or ask questions.
Business Rule	The user must enter complete and valid query details before submission.
Dependencies	FR-4
Priority	Medium

2.3.6 Functional Requirement 6

Table 2.8: Functional Requirement 6

Identifier	FR-6
Title	Helpdesk Response System
Requirement	The admin shall be able to view, manage and respond to user queries and the users shall be notified after receiving a response.
Source	Business Requirement
Rationale	The system ensures the proper communication and issue resolution.
Business Rule	• The admin must respond within 24 hours. • The query status must be updated (Pending, In Progress, Resolved) • Only admin can respond
Dependencies	FR-5
Priority	Medium

2.4 Non-Functional Requirements

Table 2.9: Non-Functional Requirements

Identifier	Category	Requirement Description	Module	Priority
NFR-1	Performance	App should load main interface within 2–3	All Modules	High
NFR-2	Performance	Map and location data should load within 3–5 seconds	navigation Module	High

NFR-3	Scalability	System should support multiple users simultaneously without performance issues	All Modules	High
NFR-4	Usability	Interface should be simple and user-friendly	All Modules	Medium
NFR-5	Responsiveness	App should work properly on different screen sizes (mobile, tablet)	All Modules	High
NFR-6	Security	User data (login, queries) should be securely stored and protected	All Modules	High

2.5 Use Case Analysis

Use case analysis is the process of identifying and describing how users and admin interact with the system to perform different tasks. In our project, it explains how user search locations and get route guidance and submit queries while the admin view and reply queries. It helps to understand the system requirements and overall working clearly.

2.5.1 Use case 1

Table 2.10: Use case 1

Use case ID	UC-1
Actor	User

Description	User views the campus map
Trigger	User opens the system
Pre- condition	System is loaded
Post-condition	Map is displayed
Main flow	• Open system • Map loads • Map displayed
Alternative flow	If map not loaded show error
Business flow	System loads map and display error
Assumptions	Internet available

2.5.2 Use case 2

Table 2.11: Use case 2

Use case ID	Use case-2
Use case Name	Search/view location
Actor	User
Description	User searches and views a location on map
Trigger	User enter location name
Pre- condition	Map is visible
Post-condition	Location is shown
Main flow	• Enter location • Search location • Show location
Alternative flow	If not found show error

Business flow	Input search and process and display result
Assumptions	Data is correct

2.5.3 Use case 3

Table 2.12: Use case 3

Use case ID	UC-3
Use case Name	Get route guidance
Actor	User
Description	User get route to selected location
Trigger	User selects location
Pre- condition	GPS enabled
Post-condition	Route displayed

Main flow	• Select location • Detect position • Generate route • Show route
Alternative flow	If GPS off ask permission
Business flow	Detect location and calculate route and display
Assumptions	GPS and internet available

2.5.4 Use case 4

Table 2.13: Use case 4

Use case ID	UC-4
Use case Name	Submit query
Actor	User
Description	User submit query through form

Trigger	User click submit
Pre- condition	Form is open
Post-condition	Query saved
Main flow	• Fill form • Submit • Save query • Show message
Alternative flow	If empty fields show error
Business flow	Submit and validate and store data
Assumptions	User enters valid data

2.5.5 Use case 5

Table 2.14: Use case 5

Use case ID	UC-5

Use case Name	Admin login
Actor	Admin
Description	Admin logs into system
Trigger	Admin enter login details
Pre- condition	Admin account exists
Post-condition	Admin dashboard open
Main flow	• Enter credentials • Verify • Login successful
Alternative flow	If wrong credentials show error
Business flow	Input credentials and validate and grant access
Assumptions	Admin has valid account

2.5.6 Use case 6

Table 2.15: Use case 6

Use case ID	UC-6
Use case Name	View queries
Actor	Admin
Description	Admin view user queries
Trigger	Admin open query section
Pre- condition	Admin logged in
Post-condition	Queries displayed
Main flow	• Open section • System fetches data • Show queries
Alternative flow	If no data show message

Business flow	Request and fetch and display
Assumptions	Queries exist in database

2.5.7 Use case 7

Table 2.16: Use case 7

Use case ID	UC -7
Use case Name	Reply to queries
Actor	Admin
Description	Admin rely to queries
Trigger	Admin selects a query
Pre- condition	Query available
Post-condition	Reply saved
Main flow	• Select query • Write query • Submit

	• Save response
Alternative flow	If empty reply shoe error
Business flow	Write reply and store and update
Assumptions	Admin provide valid response

2.6 Use Case Diagram 1

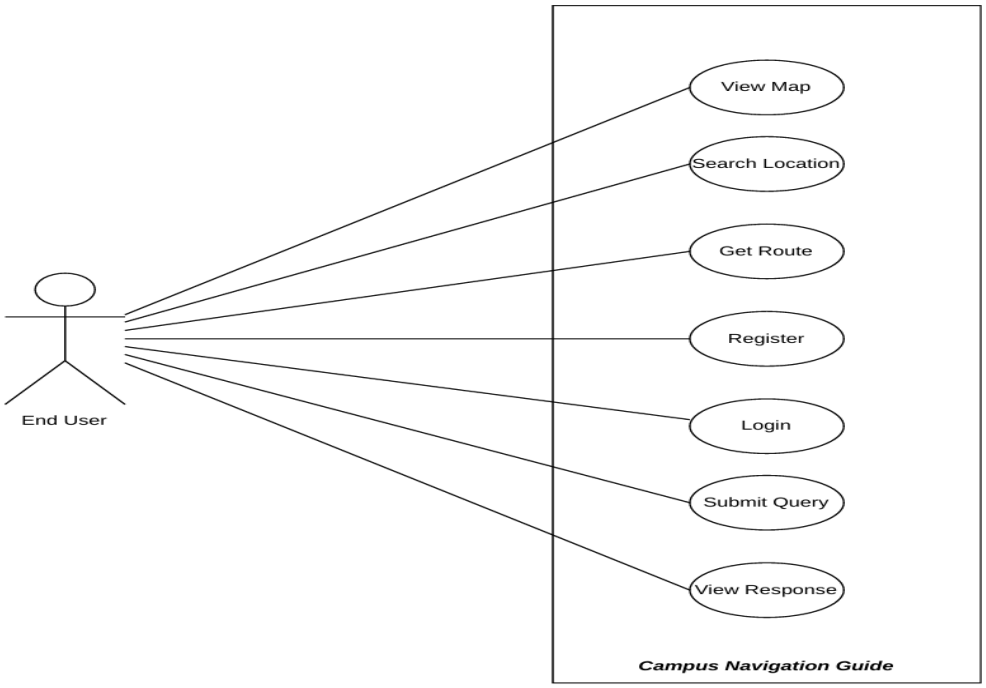


Figure 2.1: Use Case Diagram 1

2.7 Use Case Diagram 2

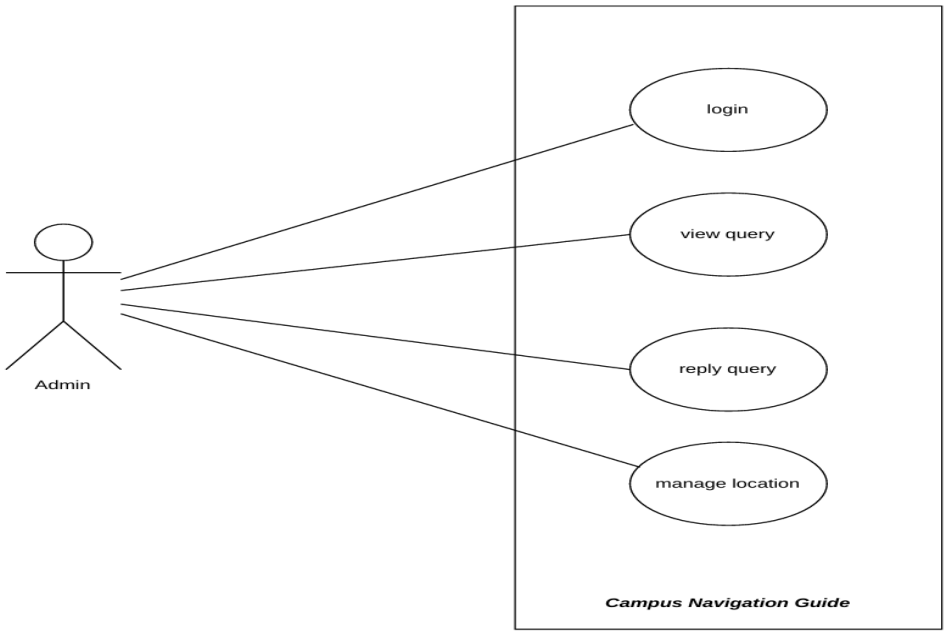


Figure 2.2: Use Case Diagram 2

Summary

In this chapter, we analyzed the requirements of the Campus Navigation Guide system. The requirements are divided into two types: functional and non-functional.

Functional requirements describe the main features of the system, such as user registration, login, searching locations, route navigation, and the help desk query system. These show how the user and admin will interact with the system. Non-functional requirements describe the quality of the system, such as performance, usability, security, and reliability. These ensure that the system works properly and is easy to use.

Use case diagrams are also included to show the interaction between the user, admin, and the system. Use case descriptions explain each function in a simple way.

Overall, this chapter helps to clearly understand the system and provides a base for the next phase, which is system design.

Chapter 3

Design and Architecture

3. System Design

Product perspective

The system will work as a web-based application running in a browser. It depends on standard web technologies (HTML, CSS, and JavaScript) for the interface and the uses Leaflet.js with OpenStreetMap to display the campus map. The backend uses PHP to process requests and MySQL to store data. The system interacts only with the user's browser and the database. No external paid service or third-party server is used, which makes the system simple and easy to maintain.

The system also depends on the browsers Geolocation API to show the users current location. If the user denies permission, only manual navigation will work. All pages connect to the backend using normal HTTP requests. The application is designed to run on the desktop and mobile device as long as the user has an internet connection.

Design Constraint

- The map must load quickly with limited internet speed.
- Route guidance depends on the accuracy of the coordinates added by the admin.
- GPS accuracy may vary based on the device quality and signal strength.
- Database queries must be optimized to keep the help desk and map features responsive.
- The system must work on common browsers like Chrome, Firefox, and Edge.

3.1 Design Consideration

Assumptions and Dependencies

- Users have access to a stable internet connection.
- Admin provides correct building coordination.
- The system depends on OpenStreetMap tiles and Leaflet.js for map display.
- The system requires PHP and MySQL support on the server.

Limitations

- Indoor navigation (inside buildings) is not supported.
- GPS location may not be accurate inside enclosed areas or basements.
- The system only covers one campus at a time.
- No advanced routing like turn-by-turn voice navigation.
- Map performance may slow down on very devices.

Risks

- Users may deny location permission, reducing GPS accuracy.
- The map may not load if OSM tile servers are temporarily slow.
- Wrong location data entered by admin can lead to incorrect routes.
- Browser incompatibility issues if older browser are used.
- Database failures or server downtime can affect help desk responses.

3.2 Requirements Traceability Matrix

Table 3.1: Requirement Traceability Matrix

Requirement ID	Requirement Description	Design Specification
R1	The system must display the campus map with building markers.	Component “Map Viewer (Leaflet + OSM)”
R2	The system must allow the users to search for campus location.	Component “Search Function+ Database Query”
R3	The system must detect and show the user’s current location.	Component “Browser Geolocation API Handler”

R4	The system must provide simple route guidance to selected locations.	Component “Route Generator (Leaflet Routing)”
R5	The system must allow users to submit help desk queries online.	Component “Help Desk Form+ Query Handler”
R6	The system must store user queries and admin replies in the database.	Component “Help Desk Database Module”
R7	The system must allow admin to view and reply to user’s queries.	Component “Admin Dashboard Query Manager”
R8	The system must let the admin add or update campus location details.	Component “Location Management Module”
R9	The system must authentication users before accessing admin features.	Component “User Login + Session Control”

3.3 Design Models

3.3.1 Class Diagram

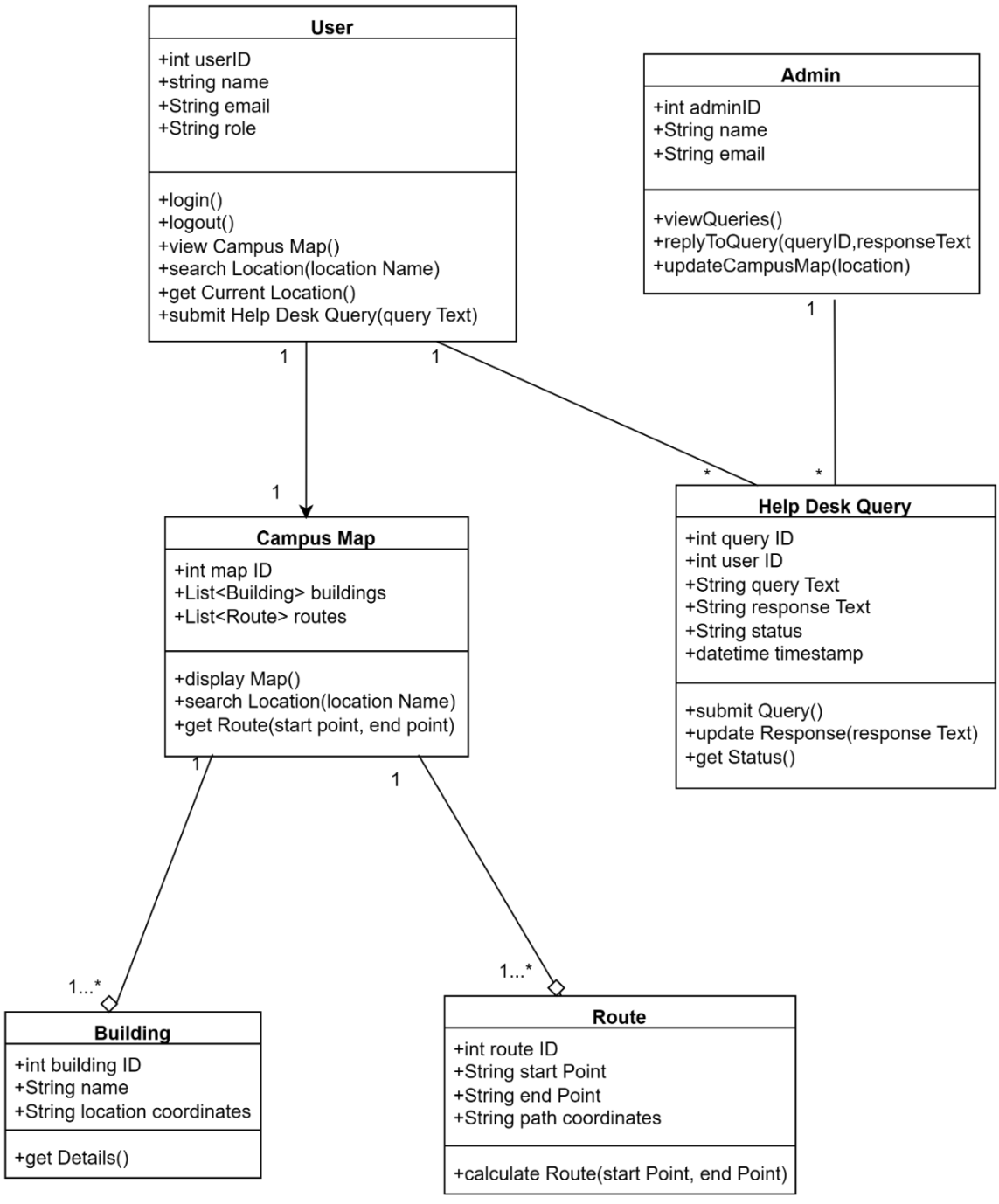


Figure 3.1: Class Diagram

3.3.2 Sequence Diagram

- **Campus Navigation Module**

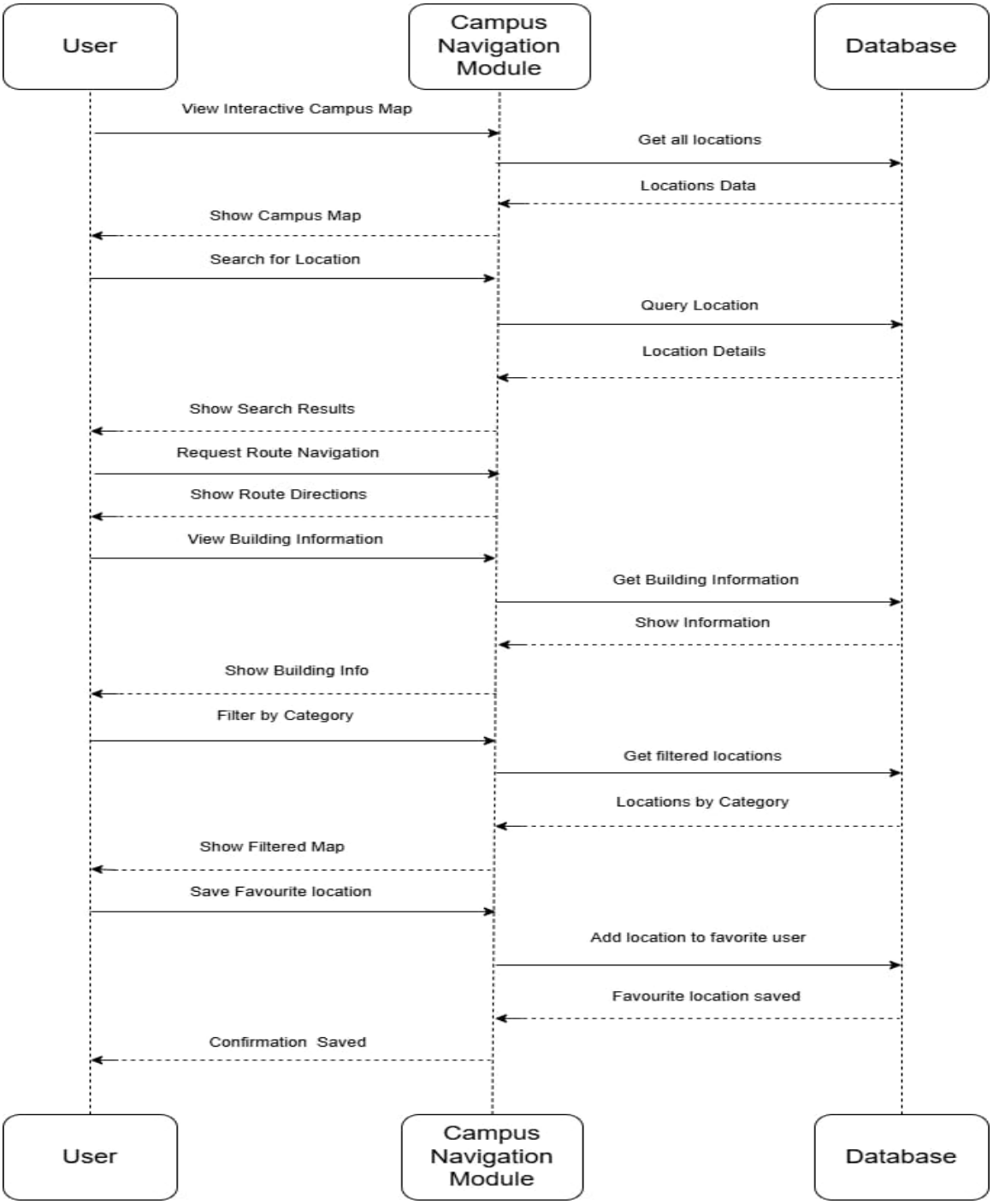


Figure 3.2: Campus Navigation Module

- **User Account Module**

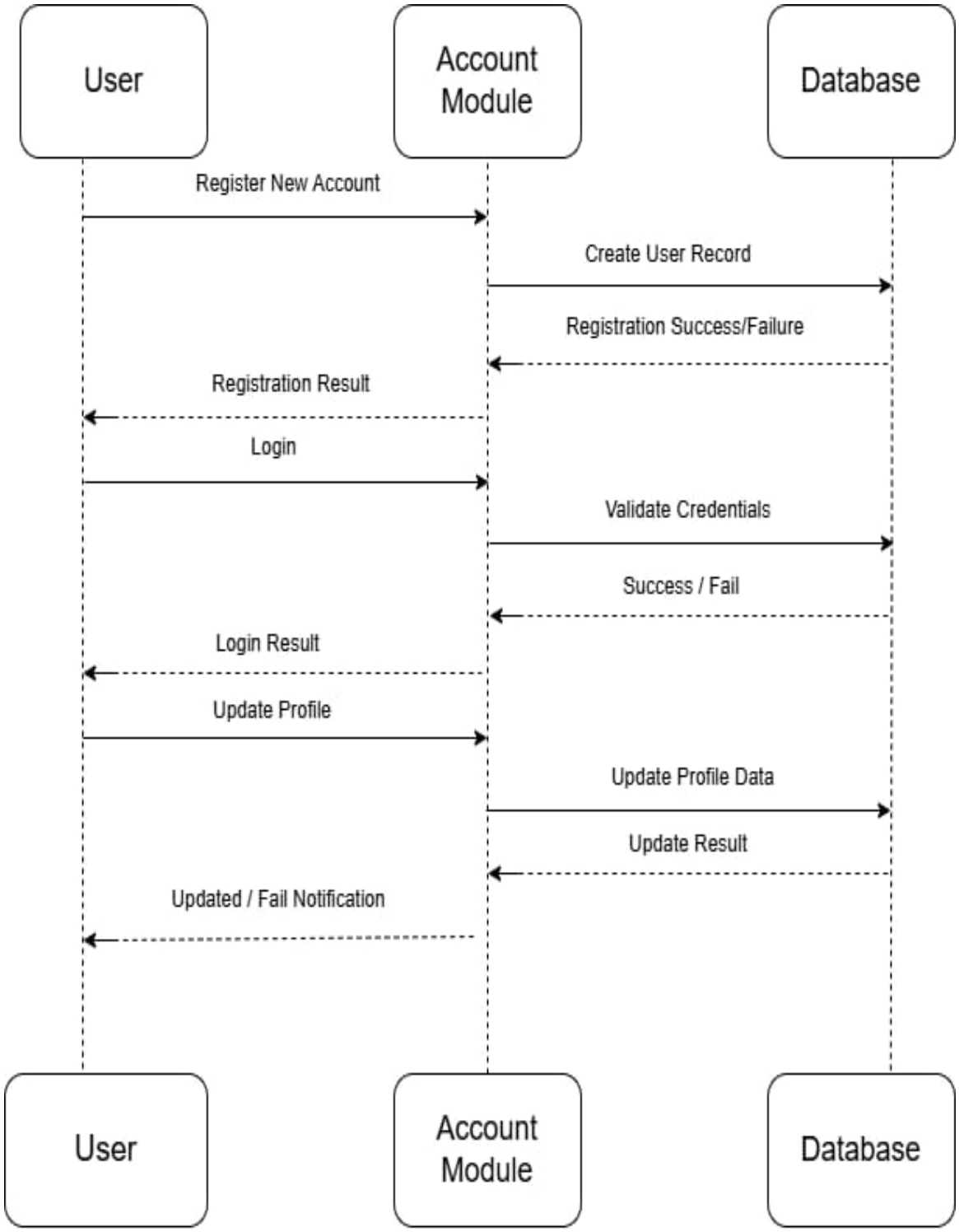


Figure 3.3: User Account Module

- **Admin Module**

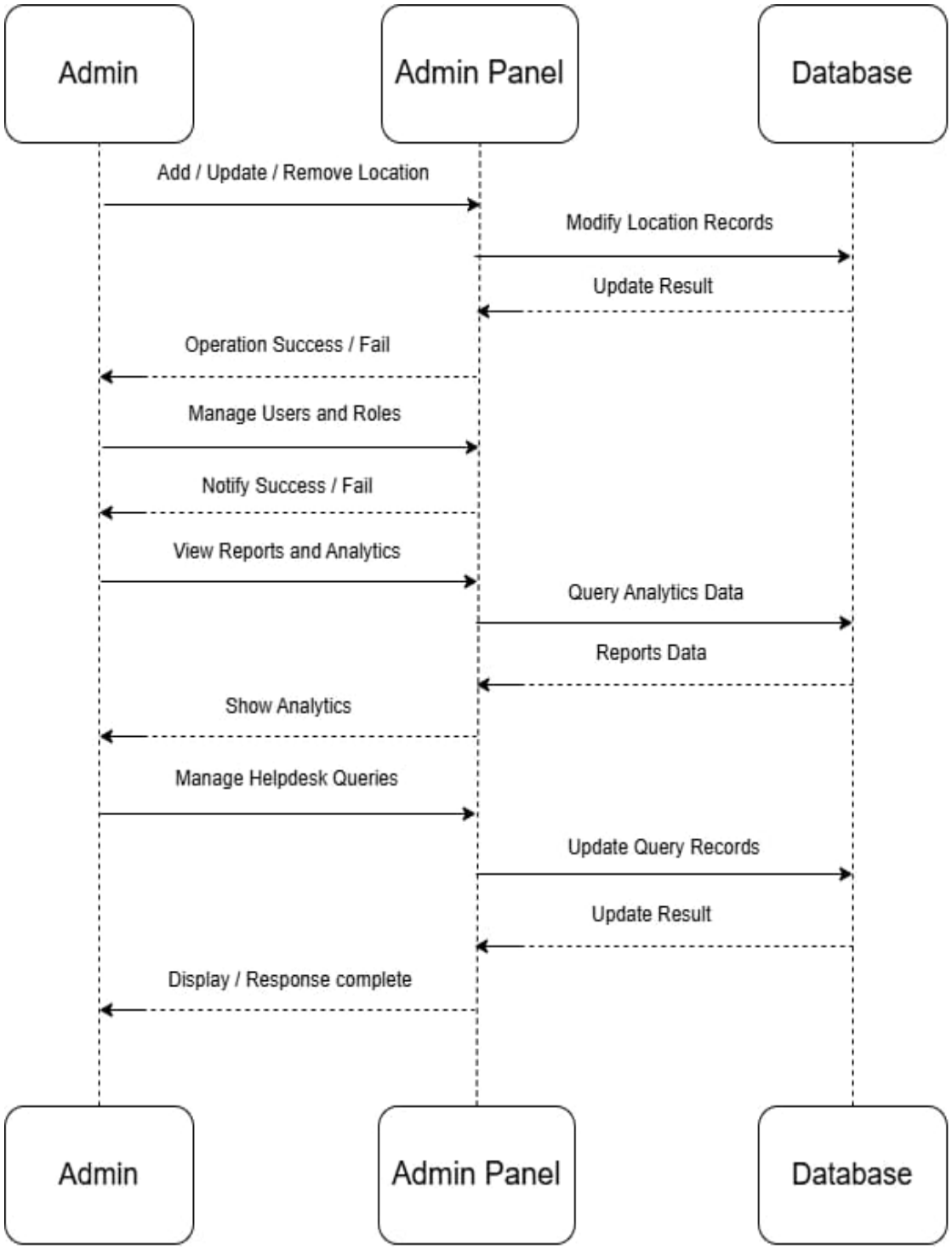


Figure 3.4: Admin Module

- **Help Desk Module**

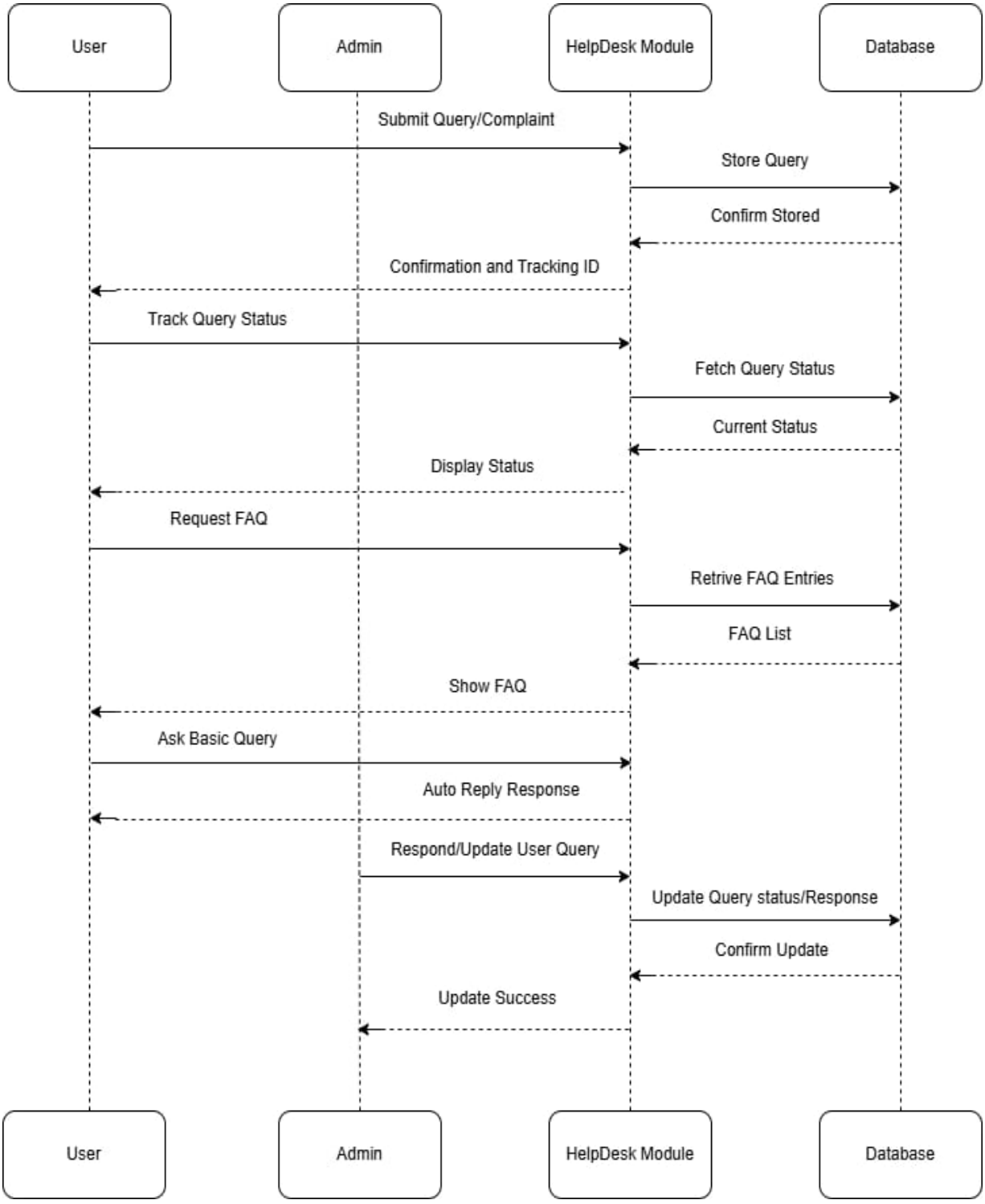


Figure 3.5: Help Desk Module

3.3.3 State Transition Diagram

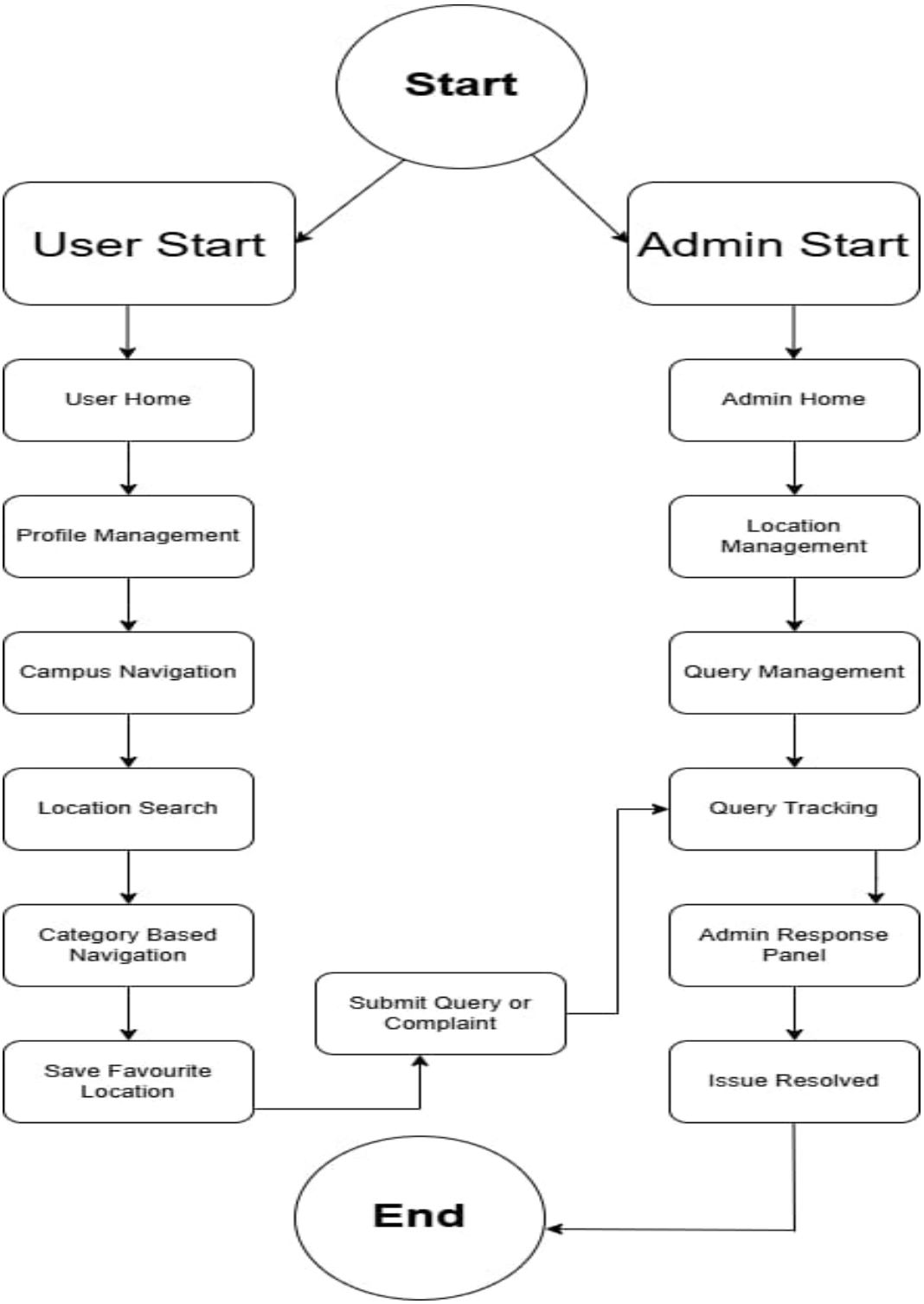


Figure 3.6: State Transition Diagram

3.4 Architectural Design

- **UML Class Diagram**

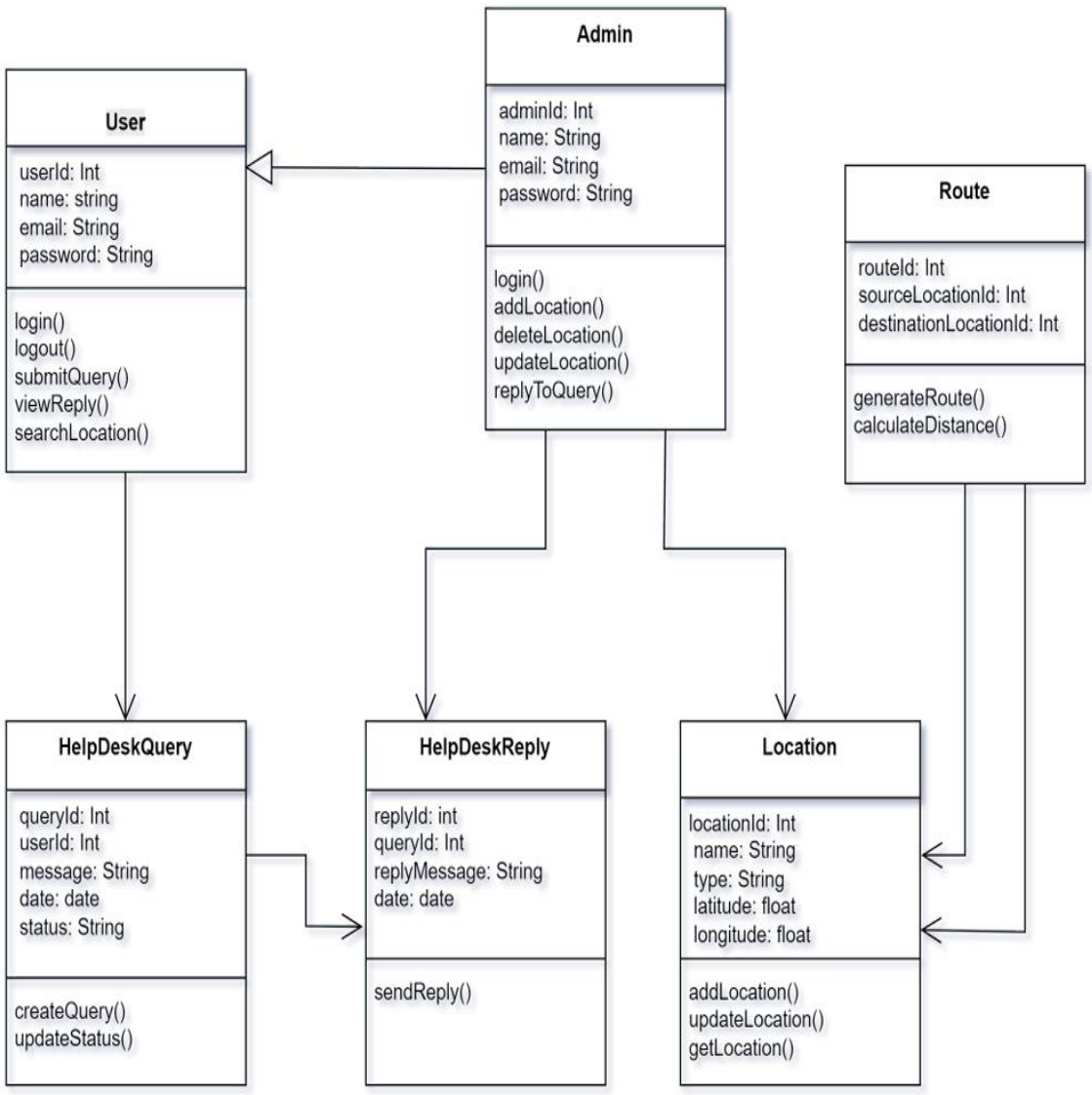


Figure 3.7: UML Class Diagram

- **UML Component Diagram**

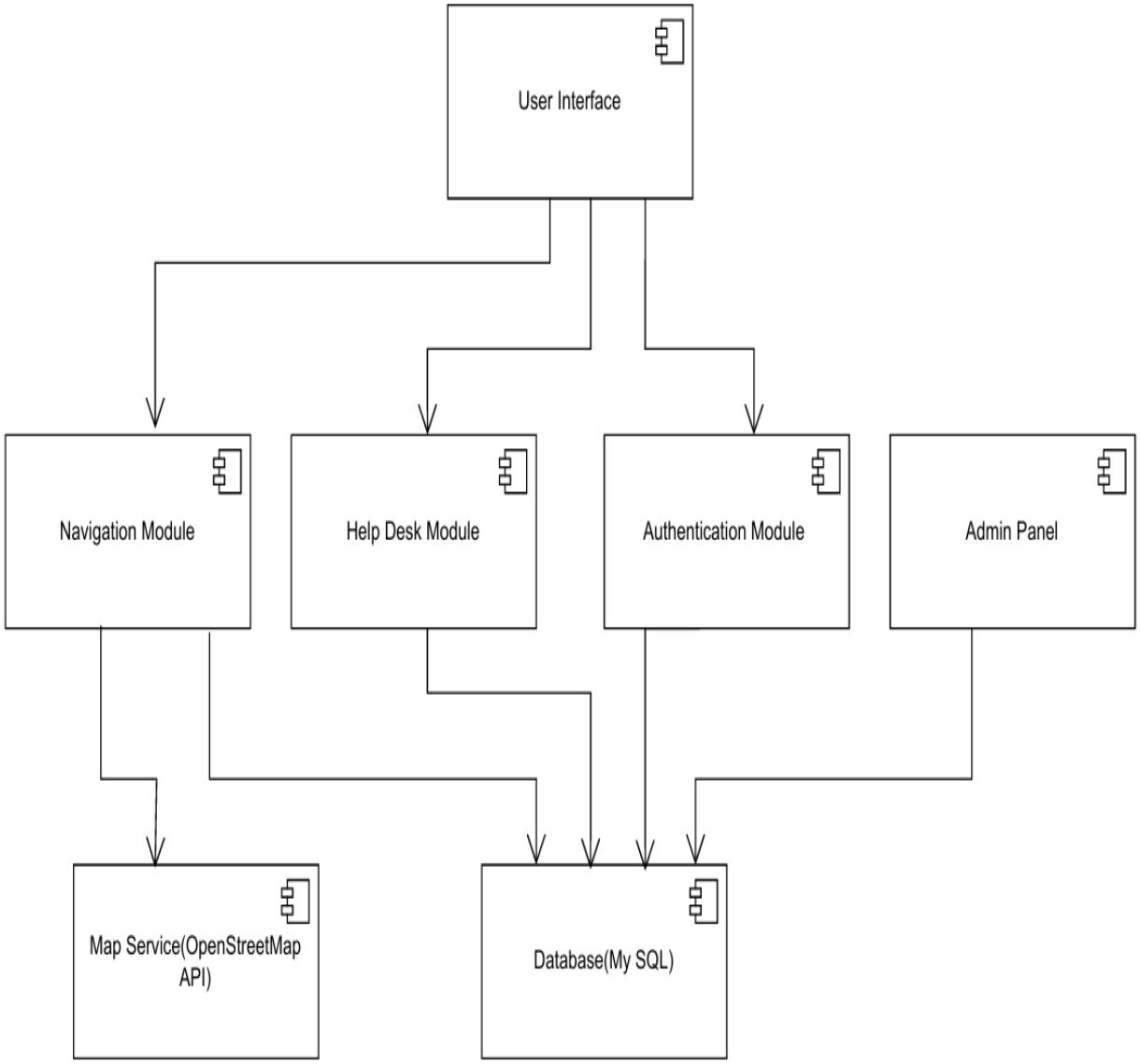


Figure 3.8: UML Component Diagram

- **ER Diagram**

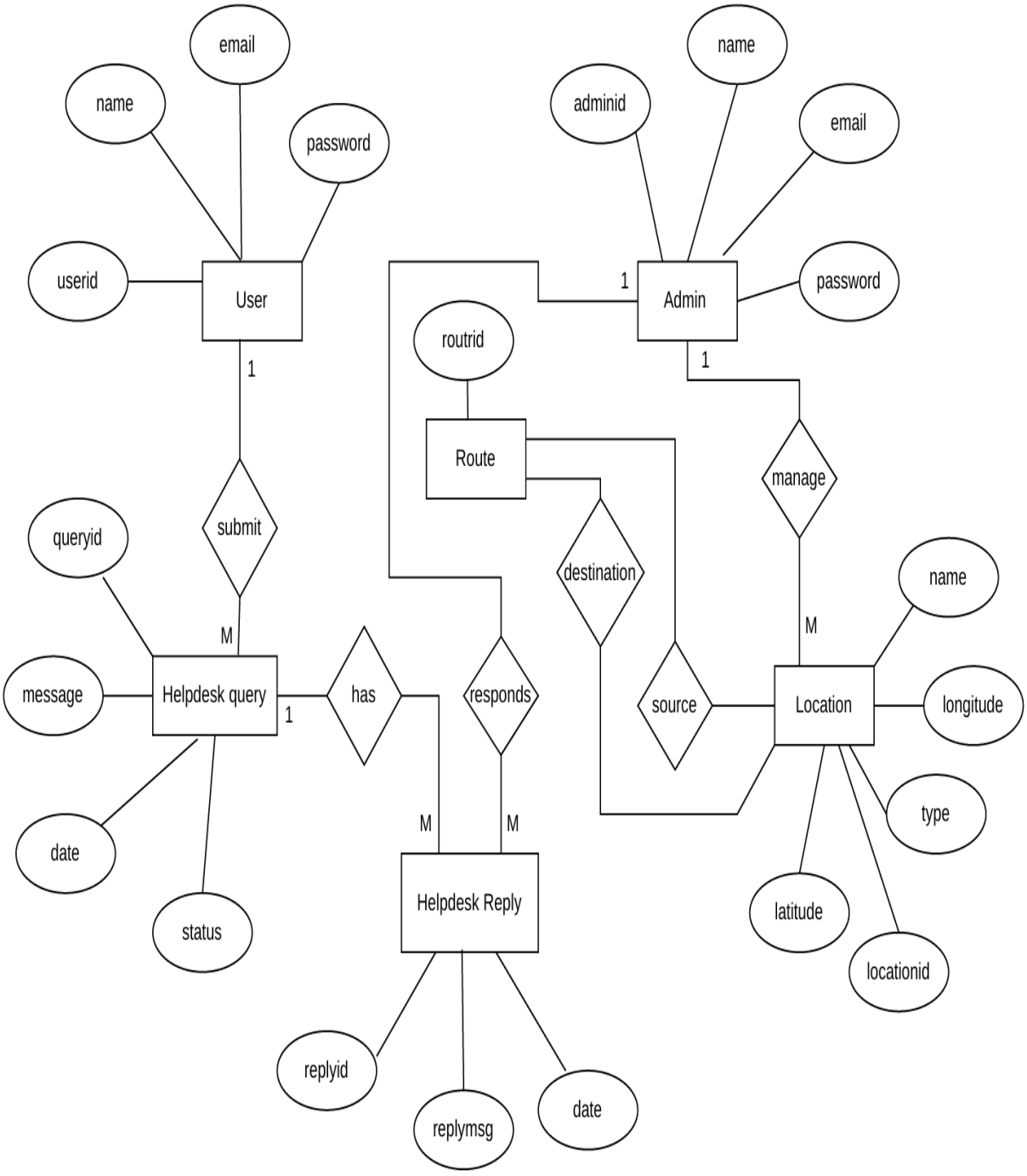


Figure 3.9: ER Diagram

3.5 Data Design

Transforming information into data structures:

Here's an explanation of how the information domain of the campus navigation and help desk system is transformed into data structures:

Campus Navigation Module

It needs to store:

- Buildings
- Departments
- Markers (points on the map such as labs, offices, gates, etc.)
- Routes (optional saved routes for users)

These are stored in tables such as:

- Buildings
- Departments
- Markers
- Routes

Each table has fields like name, latitude, longitude etc., which the system uses to display locations and generate routes.

Help Desk Module

It needs:

- User who submit a query
- Help query details
- Admin responses

These are stored in:

- Users
- Help-queries
- Help-responses
- Attachments (if the user uploads documents or images)

How the data is processed

- When a user opens the map and system fetches markers from database.
- When user searches a place and query compares search text with building/ marker names.
- When user submits a help desk message and data goes into **help-queries.**
- When admin replies and reply saved in **help-responses.**

How data is organized

All information is organized into entities (tables) Relationships include:

- One building and many departments
- One query and many admin responses
- One user and many helpdesk queries

This structure reduces data duplication and makes retrieval fast.

3.6 Data Dictionary

Table 3.2: Data Dictionary

Entity / Term	Description
User	Represents a user of the system such as a student, teacher, or visitor. It contains user login details and basic personal information.
Admin	Represents the system administrator who manages locations, routes, and replies to help desk queries.
Profile	Represents the detailed profile of a user including name, email, and account information.
Location	Represents a campus place such as a classroom, office, department, lab, or block. It contains name, type, and map coordinates.
Route	Represents the path between two locations on the campus map. It stores source and destination locations.
HelpDeskQuery	Represents a question or problem submitted by a user to the help desk. It contains the query message, date, and status.

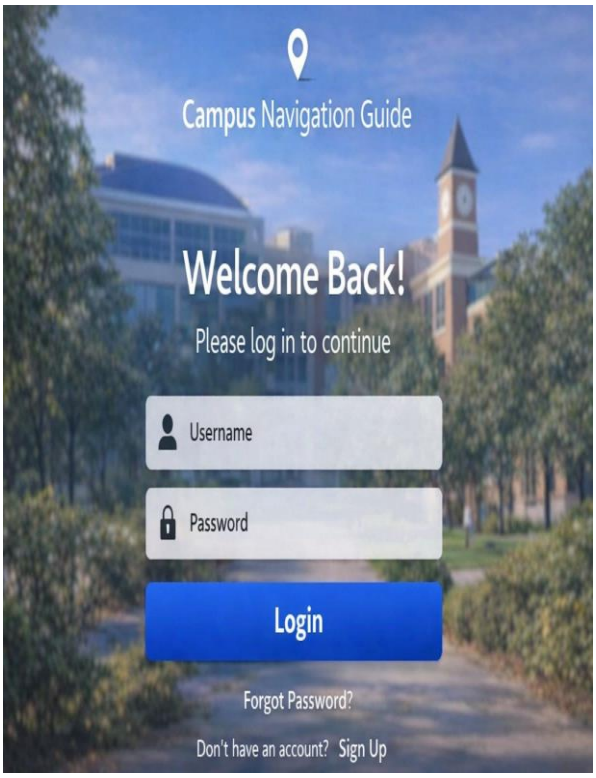
HelpDeskReply	Represents the reply given by the admin against a user query. It contains the reply message and reply date.
Authentication	Represents the login and registration process used to verify users and admins.
Navigation Module	Represents the module that provides map view, location search, and route guidance inside the campus.
Help Desk Module	Represents the module that allows users to submit queries and receive replies from the admin.
Database	Represents the system database where all users, locations, routes, and help desk records are stored.

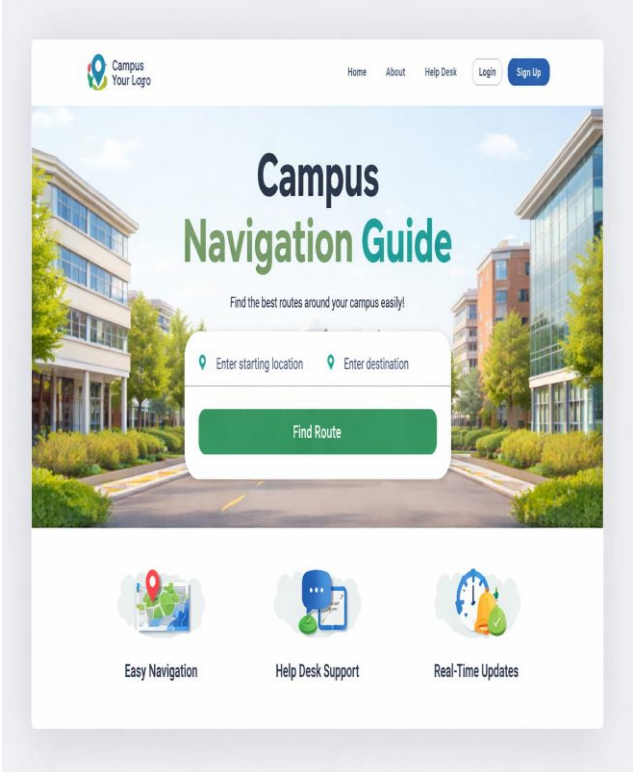
3.7 User Interface Design

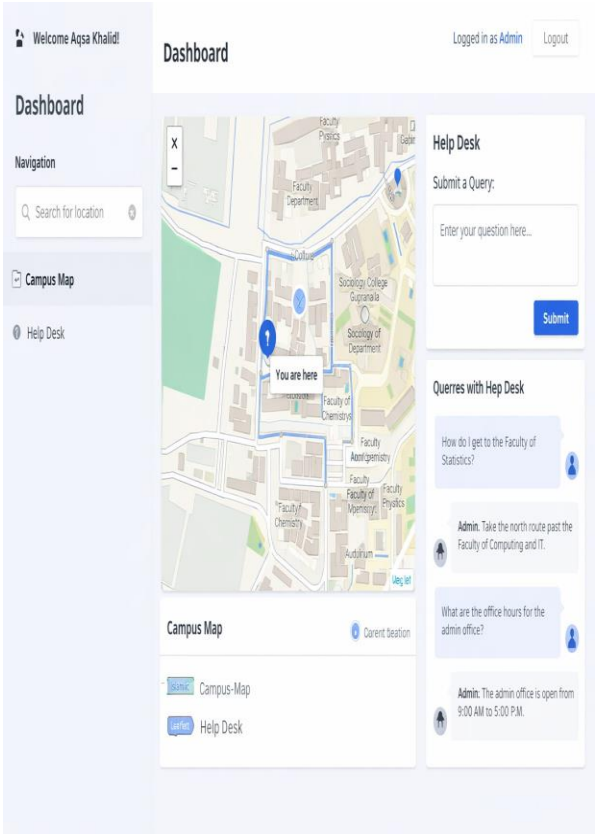
The Campus Guide System is a web-based application designed to help students, visitors, and staff easily find campus locations and get help through an online help desk. The user interface is kept simple and easy to understand so that any user can use the system without training.

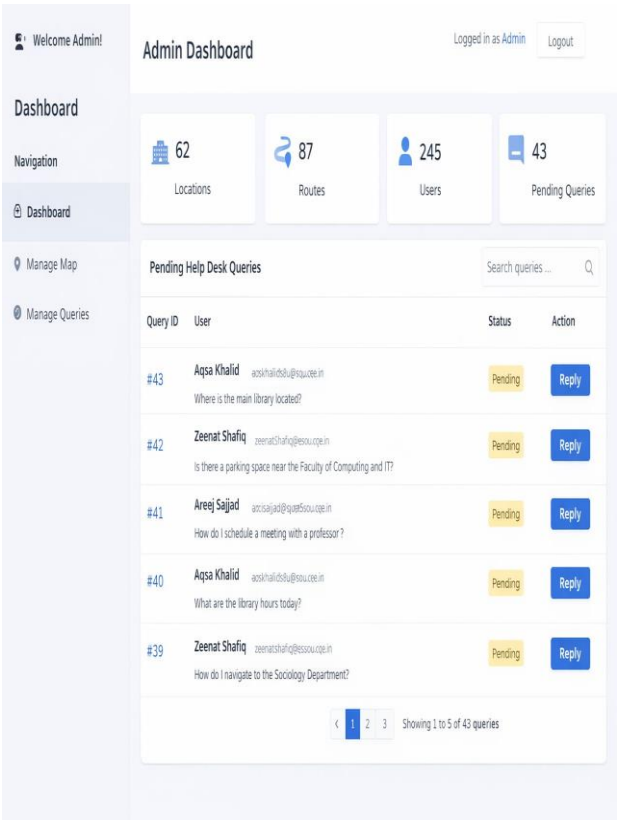
When the user opens the system a clean home page is shown with options for login, navigation, and helpdesk. After login, the user search can search for building or office using the search bar. The system shows the selected location on the map and provides the route from the user's current position to the destination. The map interface is clear and interactive so users can zoom in and out easily.

For the help desk, users can open the help desk page, write their query, and submit it. The system gives a confirmation message after successful submission. Users can also view replies from the admin in their dashboard. The admin has a separate interface where they can manage locations and reply to user queries. Overall, the interface is designed to be simple, user-friendly, and suitable for daily campus use.


Campus Navigation Guide
Welcome Back!
Please log in to continue
Username
Password
Login
Forgot Password?
Don't have an account? Sign Up


Campus Your Logo
Home
About
Help Desk
Login
Sign Up
Campus Navigation Guide
Find the best routes around your campus easily!
Enter starting location
Enter destination
Find Route
Easy Navigation
Help Desk Support
Real-Time Updates


Welcome Aqsa Khalid!
Dashboard
Navigation
Search for location
Campus Map
Help Desk
Dashboard
Logged in as Admin
Logout
Help Desk
Submit a Query:
Enter your question here...
Submit
Querres with Hep Desk
How do I get to the Faculty of Statistics?
Admin. Take the north route past the Faculty of Computing and IT.
What are the office hours for the admin office?
Admin. The admin office is open from 9:00 AM to 5:00 P.M.
You are here
Campus Map
Campus-Map
Help Desk


Welcome Admin!
Dashboard
Navigation
Dashboard
Manage Map
Manage Queries
Admin Dashboard
Logged in as Admin
Logout
62
Locations
87
Routes
245
Users
43
Pending Queries
Pending Help Desk Queries
Search queries ...
Query ID
User
Status
Action
#43
Aqsa Khalid
Where is the main library located?
Pending
Reply
#42
Zeenat Shafiq
Is there a parking space near the Faculty of Computing and IT?
Pending
Reply
#41
Areej Sajjad
How do I schedule a meeting with a professor?
Pending
Reply
#40
Aqsa Khalid
What are the library hours today?
Pending
Reply
#39
Zeenat Shafiq
How do I navigate to the Sociology Department?
Pending
Reply
Showing 1 to 5 of 43 queries

3.8 Design Decisions

Here is the design choice of our project campus navigation and help desk system:

Frontend Framework

We used html, CSS, Bootstrap, and JavaScript. Html and CSS handle page structure and styling, Bootstrap makes the interface responsive and simple, and java Script manages interactive features like map markers, location search, and route guidance. This combination is easy to implement and understand for student projects.

Backend Framework

PHP chosen for backend to process help desk forms, manage queries and interact with the database. PHP works well with XAMPP, making development easy.

Database Management System

We selected MySQL to store user data, campus locations, help desk queries, and admin responses. The database is designed in Third Normal Form (3NF) to prevent data duplication and allow easy updates.

Authentication and Authorization

Admins have a login system to access and reply to queries, while students and visitors can submit help desk requests as guests. This ensures security for admin actions and simplicity for general users.

Map and Routing

We used Leaflet.js to display an interactive campus map with building markers, and Leaflet Routing Machine for simple route guidance from the user's location to a selected destination. These tools are free, lightweight, and easy to integrate.

Error Handling and Validation

All forms and inputs are validated and sanitized on both client-side and server-side to prevent incorrect data entry and security issues like SQL injection and XSS.

Code Organization and Modularity

The code is organized into separate modules for frontend, backend, and database scripts, following a clear folder structure. This makes the system easy to maintain, update, and extend in the future.

Summary

The Campus Navigation & Help Desk System is a web-based application designed to help students, visitors, and staff easily find different locations inside the university campus. The system provides a digital map where users can search for buildings, offices, and departments and get the shortest route to reach their destination. This saves time and reduces the need to ask people for directions.

The system also includes a help desk feature where users can submit their questions online. The admin can view these queries and send replies through the system. This makes communication between users and the administration easy and fast. Overall, the project aims to make campus navigation simple and improve the help and support system using a user-friendly web application.

Chapter 4

Implementation

4. Implementation

Implementation is the phase where we convert our design into a working system. In our project, we develop the frontend using HTML, CSS, and JavaScript to create the interface and map features. Then we use PHP for backend to handle user request and MySQL to store data like queries and locations. Finally, we connect all parts and test the system to make sure everything works properly.

4.1 Algorithms

Algorithm 1 - View map

Table 4.1: Algorithm 1 - View map

Section	Details
Algorithm	View map
Input	User open system
Output	Campus map displayed

Pseudo code

```
FUNCTION ViewMap()
LoadPage()
InitializeMap()
LoadMapData()
DisplayMap()
END FUNCTION
```

Algorithm 2 – Search location

Table 4.2: Algorithm 2 – Search location

Section	Details
Algorithm	Search location
Input	Location name
Output	Location displayed on map

Pseudo code

FUNCTION SearchLocation()
INPUT locationName

```
IF locationName IS EMPTY THEN
    DISPLAY "Enter location"
ELSE
    result ← SearchDatabase(locationName)

    IF result IS FOUND THEN
        ShowOnMap(result)
    ELSE
        DISPLAY "Not found"
    END IF
END IF
```

END FUNCTION

Algorithm 3 – Route guidance

Table 4.3: Algorithm 3 – Route guidance

Section	Details
Algorithm	Route guidance
Input	Current location, destination
Output	Route displayed

Pseudo code

FUNCTION RouteGuidance()
currentLocation ← GetCurrentLocation()
destination ← GetDestination()

```
IF GPS IS ENABLED THEN
    route ← GenerateRoute(currentLocation, destination)
    DisplayRoute(route)
ELSE
    DISPLAY "Enable GPS"
END IF
```

END FUNCTION

Algorithm 4 – Submit query

Table 4.4: Algorithm 4 – Submit query

Section	Details
Algorithm	Submit query
Input	User query data
Output	Query stored in database

Pseudo code

Function submit query ()

1. Data 1 get user input ()
2. If data is valid then
3. Save to database (data)
4. Display “submitted”
5. else
6. Display “error”
7. end if

End

Algorithm 5 – Admin login

Table 4.5: Algorithm 5 – Admin login

Section	Details
Algorithm	Admin login
Input	Username, password
Output	Access granted or denied

Pseudo code

FUNCTION SubmitQuery()
data ← GetUserInput()

```
IF data IS VALID THEN
    SaveToDatabase(data)
    DISPLAY "Submitted"
ELSE
    DISPLAY "Error"
END IF
```

END FUNCTION

Algorithm 6 – View queries

Table 4.6: Algorithm 6 – View queries

Section	Details
Algorithm	View queries
Input	Admin request
Output	Queries request displayed

Pseudo code

FUNCTION ViewQueries()
queries ← FetchFromDatabase()

```
IF queries EXIST THEN
    DisplayQueries(queries)
ELSE
    DISPLAY "No data"
END IF
```

END FUNCTION

Algorithm 7 – Reply to queries

Table 4.7: Algorithm 7 – Reply to queries

Section	Details
Algorithm	Reply query
Input	Selected query, reply
Output	Response saved

Pseudo code

FUNCTION ReplyQuery()
selectedQuery ← SelectQuery()
reply ← GetInput()

```
IF reply IS NOT EMPTY THEN
    SaveReply(selectedQuery, reply)
    DISPLAY "Sent"
ELSE
    DISPLAY "Error"
END IF
```

END FUNCTION

4.2 External APIs/SDKs

Coder classroom integrates several third-party APIs and libraries to deliver its core functionality. The table below provides a detailed breakdown of each external dependency used in the project implementation.

Table 4.8: External APIs/SDKs

APIs/SDKs Name and version	**Description**	**Purpose of usage**	**Endpoints/ functions/ class used**
Leaflet.js (v1.9)	A JavaScript library for interactive maps	Used to display campus map and markers	L. map (), L. marker (), L. tile Layer ()
Open street map	Free and open source map data provider	Provide map tiles and location data	Tile layer URL integration
JavaScript fetch API	Built in JavaScript feature	Used for communication between frontend and backend	Fetch (), GET/POST requests
PHP	Server-side scripting language	Handles backend logic and form processing	$-POST, MySQL-query ()
MySQL	Database management system	Stores location, queries, and admin data	SELECT, INSERT, UPDATE queries

4.3 Code repository

Git is used to manage the project code and track changes. All files including frontend (HTML, CSS, and JavaScript), backend (PHP), and database (MySQL) are stored in the repository. It helps team members to collaborate and maintain the latest version of the project.

GitHub repository link: https://github.com/areejsajjad123/Campus-Navigation-Guide.git

4.3.1 Matrices of the git repository

The following metrics are monitored to ensure effective use of the git repository.

Commits: The number of commits made to track the progress and consistency of contributions by team members. Each member regularly commits their work.

Branches: Different branches are created for separate features such as map module, help desk module, and admin panel. This helps in organized development.

Pull requests: Pull requests are used to merge changes from different branches into the main project. This ensures proper review before adding new features.

Issues: Issues are used to track bugs, errors, and feature request. Team members can assign and manage tasks easily.

Contributors: All group members are added as contributors, and their work is tracked through commits and updates.

Code Reviews: Code reviews are performed before merging any changes to maintain code quality and ensure proper functionality of the system.

Summary

In this section, we implemented our system using HTML, CSS, JavaScript, PHP, and MySQL. We created frontend, backend, and database to make the system functional. Algorithms were used to explain system working and external tools like leaflet.js were used for map features. All project code is stored in a repository for management.

Chapter 5

Testing and Evaluation

5. Testing and Evaluation

This chapter focuses on testing and evaluation of our project. The validation and verification of different components will be considered.

5.1 Unit Testing

5.1.1 Unit Test Case 1: User Login

Table 5.1: Unit Test Case 1: User Login

Field	Details
Test Case ID	UT1
Module	User Account
Title	User Login with valid credentials
Description	This test verifies that a registered email can successfully log into the system using correct email and password.
Objective	Verify that a user can log in with valid credentials.
Preconditions	User account must be already registered in the system.
Input	Valid email and password
Test steps	1. Open login page 2. Enter valid credentials 3. Click login button
Expected Result	User should be redirected to dashboard
Actual Result	User successfully logged in and redirected
Remarks	Pass

5.1.2 Unit Test Case 2: Search Location

Table 5.2: Unit Test Case 2: Search Location

Field	Details
Test Case ID	UT2
Module	Navigation
Title	Search Location Functionality
Description	This test verifies that a user can search campus location and system returns accurate result.
Objective	Verify that the system returns the correct location for a search query.
Preconditions	Map API should be active
Input	Computer Lab
Test Steps	1. Enter location in search bar 2. Click search
Excepted Result	Correct location displayed on map
Actual Result	Exact location shown with marker
Remarks	Pass

5.1.3 Unit Test Case 3: Submit Help Desk Query

Table 5.3: Unit Test Case 3: Submit Help Desk Query

Field	Details
Test Case ID	UT3
Module	Help Desk
Title	Submit User Query
Description	This test ensures that a user can submit query through help desk.

Objective	Verify that a user can submit a query and it is stored in the database.
Preconditions	User must be logged in
Input	Where is Admin Office?
Test Steps	1. Open help desk 2. Enter query 3. Click submit
Excepted Result	Query should be saved in database
Actual Result	Query stored successfully
Remarks	Pass

5.1.4 Unit Test Case 4: Admin Reply

Table 5.4: Unit Test Case 4: Admin Reply

Field	**Details**
Test Case ID	UT4
Module	Admin
Title	Admin Reply to User Query
Description	This test verifies that admin can reply to user queries and response is stored.
Objective	Verify that admin can reply to queries and response is saved.
Preconditions	Query must exist
Input	Reply message
Test Steps	1. Admin opens query 2. Writes reply 3. Click submit
Excepted Result	Reply saved in database

Actual Result	Reply stored successfully
Remarks	Pass

5.2 Functional Testing

5.2.1 Functional Test Case 1: Search Location

Table 5.5: Functional Test Case 1: Search Location

Field	**Details**
Test Case ID	FT1
Function	Search Location
Objective	Verify system to return correct location results.
Description	This test ensure that user can search for any location and relevant result is displayed.
Preconditions	System connected to map or database
Input	Location search (Computer Lab)
Test Steps	1. Enter location 2. Click search
Expected Result	Correct location displayed
Actual Result	Location shown correctly
Remarks	Pass

5.2.2 Functional Test Case 2: Map Display

Table 5.6: Functional Test Case 2: Map Display

Field	**Details**
Test Case ID	FT2

Function	Map Display
Objective	Verify map loads correctly with location markers
Description	This test checks that map is displayed properly with markers for locations
Preconditions	Internet connection available
Input	User opens map interface
Test Steps	1. Open map screen
Expected Result	Map loads with markers
Actual Result	Map loaded successfully
Remarks	Pass

5.2.3 Functional Test Case 3: Route Generation

Table 5.7: Functional Test Case 3: Route Generation

Field	**Details**
Test Case ID	FT3
Function	Route Navigation
Objective	Verify system to generate shortest route between locations.
Description	This test ensure that user can select source and destination and get optimal route.
Preconditions	Locations must exist
Input	Source to Destination
Test Steps	1. Select location 2. Click generate route
Expected Result	Shortest route displayed

Actual Result	Route generated correctly
Remarks	Pass

5.2.4 Functional Test Case 4: Help Desk System

Table 5.8: Functional Test Case 4: Help Desk System

Field	**Details**
Test Case ID	FT4
Function	Help Desk
Objective	Verify user can submit queries and receive replies.
Description	This test ensure complete query submission and admin response functionality works properly.
Preconditions	User must be logged in
Input	Query message
Test Steps	1. Submit query 2. Check reply
Expected Result	Query stored and reply received
Actual Result	Working correctly
Remarks	Pass

5.3 Integration Testing

5.3.1 Integration Test Case 1: Login + Dashboard

Table 5.9: Integration Test Case 1: Login + Dashboard

Field	**Details**
Test Case ID	IT1

Module	Login + Dashboard
Title	Login and Dashboard Integration
Description	This test verifies that user is redirected to dashboard after login.
Objective	Verify that users are redirected to the dashboard after login.
Preconditions	User account exists
Input	Valid login credentials
Test Steps	1. Login with valid details
Expected Result	Dashboard should open
Actual Result	Dashboard displayed
Remarks	Pass

5.3.2 Integration Test Case 2: Search + Map Display

Table 5.10: Integration Test Case 2: Search + Map Display

Field	**Details**
Test Case ID	IT2
Module	Search + Map
Title	Search and Map Integration
Description	This test verifies that searched location is correctly displayed on map
Objective	Verify that searched locations are correctly displayed on the map.
Preconditions	Map service running
Input	Location name
Test Steps	1. Enter location 2. Click search
Expected Result	Marker should show on map

Actual Result	Marker displayed correctly
Remarks	Pass

5.3.3 Integration Test Case 3: Search + Route Navigation

Table 5.11: Integration Test Case 3: Search + Route Navigation

Field	**Details**
Test Case ID	IT3
Module	Search + Navigation
Title	Search and Route Integration
Description	This test verifies that searched location can be used to generate route.
Objective	Verify that searched locations can be used to generate routes.
Preconditions	Locations must exist
Input	Source to Destination
Test Steps	1. Search location 2. Generate route
Excepted Result	Route should be displayed
Actual Result	Route shown correctly
Remarks	Pass

5.3.4 Integration Test Case 4: Help Desk + Database

Table 5.12: Integration Test Case 4: Help Desk + Database

Field	**Details**
Test Case ID	IT4
Module	Help Desk + Database

Title	Query Storage Integration
Description	This test verifies that submitted query is stored in database.
Objective	Verify that user queries are stored in the database.
Preconditions	Database connected
Input	Query message
Test Steps	1. Submit query
Expected Result	Data stored in database
Actual Result	Successfully stored
Remarks	Pass

5.3.5 Integration Test Case 5: Admin + Help Desk

Table 5.13: Integration Test Case 5: Admin + Help Desk

Field	**Details**
Test Case ID	IT5
Module	Admin + Help Desk
Title	Admin Response Integration
Description	This test verifies that admin reply is visible to user.
Objective	Verify that admin replies are visible to users.
Preconditions	Query must exist
Input	Reply
Test Steps	1. Admin replies 2. User checks dashboard
Expected Result	Reply should be visible
Actual Result	Reply shown

Remarks	Pass

5.4 Performance Testing

Tool Used: Postman Collection Runner

Backend: Node.js server with database

Performance Testing Results Table

Table 5.14: Performance Testing Results Table

Tool / Method	Metric Measured	Result / Observation
Postman (Map API)	Average map load time	2 seconds, within acceptable range (<3 sec)
Postman (Route API)	Route generation time	1.5 seconds, fast response
Postman (Help Desk API)	Query submission time	0.8 seconds, efficient
Postman (Database Query)	Data fetch response time	0.6 seconds, very fast
Postman (Concurrent Users)	System stability under load	No crashes, stable with 50 users
Browser Testing	UI responsiveness	Smooth and responsive

5.4.1 Performance Test Case 1: Map Load Time

Table 5.15: Performance Test Case 1: Map Load Time

Field	Details
Test Case ID	PT1
Title	Map Loading Speed
Description	Measure time required to load map interface
Objective	Verify map loads within acceptable time

Precondition	Internet connection available
Test Steps	Open map screen
Expected Result	Map loads in < 3 seconds
Actual Result	Loaded in 2 seconds
Remarks	Pass

5.4.2 Performance Test Case 2: Route Generation Time

Table 5.16: Performance Test Case 2: Route Generation Time

Field	**Details**
Test Case ID	PT2
Title	Route Generation Speed
Description	Measure time required to generate route
Objective	Verify route is generated quickly
Precondition	Locations available
Input	Source to Destination
Test Steps	Select locations and generate route
Expected Result	Route generated in < 2 seconds
Actual Result	Generated in 1.5 seconds
Remarks	Pass

5.4.3 Performance Test Case 3: Database Response Time

Table 5.17: Performance Test Case 3: Database Response Time

Field	**Details**
Test Case ID	PT3

Title	Database Response Time
Description	Measure time taken to fetch or store data
Objective	Verify fast database response
Precondition	Database connected
Test Steps	Send request to database
Expected Result	Response in < 1 second
Actual Result	0.6 seconds
Remarks	Pass

5.4.4 Performance Test Case 4: Help Desk Response Time

Table 5.18: Performance Test Case 4: Help Desk Response Time

Field	**Details**
Test Case ID	PT4
Title	Help Desk Query Response
Description	Measure response time for query submission
Objective	Verify query is processed quickly
Precondition	User logged in
Input	Query message
Test Steps	Submit query
Expected Result	Response in < 1 second
Actual Result	0.8 seconds
Remarks	Pass

Summary

The system was tested using main testing approaches: Unit Testing, Integration Testing, Functional Testing and Performance Testing. Each testing phase was conducted to ensure the system works correctly, efficiently and reliably. The system performed well in all testing phases showing correct functionality, proper integration between modules, fast response time and stable performance.

Chapter 6

System Conversion

6. System Conversion

System conversion is the process of transitioning from an existing information system to a new one. It involves shifting data, processes and user operations from the legacy environment to the newly developed system. Effective system conversion ensures minimal disruption, data integrity and seamless user adaptation.

6.1 Conversion Method

For this project, Direct Conversion method was adopted. This method involves immediately implementing the new system without running any previous system in parallel. Since this project was developed from scratch and there was no existing manual or automated system for campus navigation and help desk services, direct implementation of the new web-based system was practical and efficient. The system was deployed using HTML, CSS, JavaScript, PHP and MySQL without needing any data migration from a legacy system.

Although direct conversion can involve risks in large scale real-world systems, it was suitable for this academic project as it allowed quick development, testing and deployment.

6.2 Deployment

Deployment involves transferring the Campus Navigation Guide System from development to a live environment for user access. It includes setting up the server and database, integrating map APIs and configuring user authentication. After testing for functionality and compatibility, the system was successfully deployed and made available for users to access navigation, search and help desk features.

6.2.1 Deployment Steps

Table 6.1: Deployment Steps

Step No	Task	Details
1	Backend Setup	Setup and configure project environment
2	Database Configuration	Create and connect database for storing user, location and query data
3	Map API Integration	Configure map services to display campus locations and routes

4	Authentication Setup	Implement user login system for secure access
5	Code Deployment	Deploy application code to server or hosting platform
6	System Testing	Perform final testing to ensure all work features correctly
7	Browser Testing	Test system on different devices and browsers for compatibility
8	Post-Deployment Monitoring	Monitor system performance and fix runtime issues

6.2.2 Data Conversion

Table 6.2: Data Conversion

Step No	Task	Details
1	Data Collection	Collected campus location and route data
2	Data Cleaning	Removed duplicate and incorrect data entries
3	Data Validation	Verified accuracy and completeness of data
4	Data Transformation	Converted raw data into system-compatible format
5	Data Testing	Checked if data is correctly stored and retrieved

6.2.3 Training

Training Instruction 1: Navigation

Table 6.3: Training Instruction 1: Navigation

Step	Instruction
1	Open web application in browser using server URL
2	Allow location access to enable GPS functionality

3	View campus map with available building markers
4	Use search bar to find a specific department or location
5	Select a location to view its details and route on map

Training Instruction 2: Route Guidance

Table 6.4: Training Instruction 2: Route Guidance

Step	Instruction
1	Select current location or allow GPS detection
2	Search and choose the destination location
3	Click on “Get Route” option
4	View the route path displayed on the map
5	Follow the route visually to reach the destination

Training Instruction 3: Help Desk

Table 6.5: Training Instruction 3: Help Desk

Step	Instruction
1	Navigate to the Help Desk section
2	Enter your query or problem in form
3	Submit query
4	Wait for admin response
5	View reply in system dashboard

6.3 Post Deployment Testing

Post deployment testing was conducted to ensure that Campus Navigation Guide system is properly configured and functioning as expected. The purpose of this testing is to verify the system performance, data accuracy and user interaction after deployment.

6.3.1 Post Deployment Testing Steps

Table 6.6: Post Deployment Testing Steps

Step	Test Activity	Expected Outcome
1	Launch Application	Application opens successfully on browser using server without errors
2	Map Loading	Campus map loads correctly with all markers displayed
3	Search Functionality	User can search any building or department and get accurate results
4	GPS Location Detection	System detects and shows user's current location
5	Route Generation	Route is correctly displayed between source and destination locations
6	Help Desk Submission	User can submit queries and data is stored in the database successfully
7	Admin Reply Function	Admin can view queries and send replies correctly

6.4 Challenges and Solutions

During the development and deployment of the Campus Navigation Guide system, several technical and user-related challenges were encountered. The following table summarizes the key challenges and their implemented solutions:

Table 6.7: Challenges and Solutions

Challenges	Description	Solution Implemented
Map Integration Issues	Difficulty in integrating map API and displaying accurate campus locations	Configured map API correctly and used markers to show precise locations
Slow Map Loading	Map took longer time to load on slow internet connections	Optimized API calls and reduced unnecessary data loading
Incorrect Search Results	Search sometimes returned irrelevant or no results	Improved search logic and added proper keywords for locations
Route Generation Errors	System failed to generate correct routes between locations	Fixed path algorithm and validated source or destination inputs
Database Connectivity Issues	Data not properly stored or retrieved from database	Ensured stable database connection and optimized queries
Help Desk Query Delay	Delay in submitting or retrieving user queries	Improved backend response and query handling mechanism
User Interface Complexity	Users found Interface difficult to understand quickly	Simplified UI and improved layout design
Multiple Users Load	System slowed down when multiple users accessed simultaneously	Optimized backend performance and handled concurrent requests
Login / Authentication Issues	Some users faced login errors	Fixed validation and improved authentication logic

Summary

This chapter explains the testing and deployment of the Campus Navigation Guide system. It include system conversion, data conversion, deployment steps and how the system was setup for users. It also highlights the challenges faced and their solutions.

Chapter 7

Conclusion

7. Introduction

This chapter conclude the system by evaluating the objectives defined at the start of the project and highlight future improvement. It checks whether the system requirement have been successfully implemented and provides an overview of system performance.

7.2 Evaluation

Below is a list of objectives stated in chapter 1 and their implementation status in the final project.

Table 7.1: Evaluation

Objectives	Status
User can view campus map	✔ completed
User can search locations	✔ completed
User can get route guidance	✔ completed
User can submit help desk query	✔ completed
Admin can login to system	✔ completed
Admin can view queries	✔ completed
Admin can reply to queries	✔ completed
System provide simple and user friendly interface	✔ completed

7.2 Traceability matrix

The traceability matrix ensure that all software requirements are fulfilled by mapping them to design code and test cases.

Table 7.2: Traceability matrix

Requirement ID	Requirement description	Design specification	Code	Test ID
R1	View campus map	Map module	Map.js	UT1
R2	Search location	Search component	Search.js	FT1
R3	Get route guidance	Pouting module	Route.js	FT2
R4	Submit query	Help desk form	Submit Query. php	IT1
R5	Admin login	Auto module	Login. php	UT2
R6	View queries	Admin panel	View queries. php	FT3
R7	Reply to queries	Response module	Reply. php	IT2

7.3 Conclusion

The system successfully fulfills its main objectives. It provides an easy and efficient way for users to navigate the campus and get help through the help desk system. The system is simple user friendly and reduces confusion for students and visitors. Overall the project meets the requirements defined in the proposal.

7.4 Future work

- Develop mobile application version
- Add AI based chatbot for instant help
- Provide indoor navigation
- Add support for multiple campuses
- Improve UI design and system performance

References

Books

Ian Sommerville, Software Engineering, 10th Edition, Pearson Education, 2015.

Web Resources

Leaflet.js Documentation, “Interactive Maps Library.”

Available: https://leafletjs.com

OpenStreetMap, “Free Editable Map Data.”

Available: https://www.openstreetmap.org

PHP Official Documentation.

Available: https://www.php.net/docs.php

MySQL Documentation.

Available: https://dev.mysql.com/doc

Bootstrap Documentation.

Available: https://getbootstrap.com/docs

Mozilla Developer Network (MDN), “JavaScript Guide.”

Available: https://developer.mozilla.org

APIs / Libraries

Leaflet Routing Machine Documentation.

Available: https://www.liedman.net/leaflet-routing-machine/

Browser Geolocation API (MDN).

Available: https://developer.mozilla.org/en-US/docs/Web/API/Geolocation_API

Appendix

Appendix A: General Coding Standards & Guidelines

To ensure maintainability, readability, and scalability of the Campus Navigation Guide System, the following coding standards were followed:

A.1 Variable Naming Conventions

camelCase → variables and functions

(e.g., userName, getLocation())

PascalCase → class names

(e.g., UserModule, AdminPanel)

snake_case → database fields

(e.g., user_id, location_name)

UPPER_CASE → constants

(e.g., MAX_USERS)

A.2 Code Structure

Separate modules for:

Navigation system

Help desk system

Admin panel

Frontend, backend, and database are organized in separate files.

A.3 Input Validation & Security

Validation applied on:

Client-side (JavaScript)

Server-side (PHP)

Security measures:

SQL Injection prevention

Input sanitization

A.4 Error Handling

User-friendly error messages displayed

Backend errors handled securely

A.5 Database Guidelines

Tables normalized (3NF)

Proper relationships maintained

Efficient queries used

Appendix B: System Design and Functional Guidelines

1. System Overview

- The Campus Navigation Guide System is a web-based application designed to assist users in locating campus buildings and departments efficiently.
- The system provides map-based navigation along with a help desk support feature.

2. Core Functional Modules

The system is divided into the following main modules:

- Navigation Module: Displays campus map and route guidance
- Help Desk Module: Allows users to submit queries
- Admin Module: Enables admin to manage data and respond to queries
- User Module: Provides search and navigation functionality

3. User Interaction Flow

The system follows a structured user interaction process:

- User opens the system and views the map
- User searches for a location
- System displays the selected location
- Route is generated from current location to destination
- User may submit queries via help desk

4. Input and Output Handling

- Input is taken through forms and search fields
- Output is displayed in the form of:

1. Map markers
2. Route paths
3. Query responses

5. Validation and Data Integrity

- All user inputs are validated before processing

- Required fields must be filled before submission
- Invalid inputs generate appropriate error messages

6. System Constraints

- Requires internet connection for map loading
- GPS accuracy depends on user device
- System is limited to a single campus

7. Performance Considerations

- Map loading optimized for faster response
- Database queries designed for efficiency
- System supports multiple users with minimal delay

8. Security Considerations

- User inputs are sanitized
- Admin access is restricted through authentication
- Data is securely stored in the database

9. Usability and Interface

- Simple and user-friendly interface
- Easy navigation between pages
- Responsive design for different screen sizes

10. System Limitations

- No indoor navigation support
- Depends on internet availability
- Limited advanced routing features

Appendix C: Database and Data Management Guidelines

1. Database Overview

- The system uses MySQL database to store and manage all application data.
- Data is organized into structured tables for efficient storage and retrieval.

2. Main Database Entities

- The following tables are used in the system:

1. Users: Stores user details and login information
2. Locations: Stores campus buildings and coordinates
3. Routes: Stores path information between locations
4. Queries: Stores help desk queries submitted by users
5. Replies: Stores admin responses to queries

3. Data Relationships

- Relationships between tables are maintained:

1. One user → multiple queries
2. One query → one or more replies
3. Locations linked with routes

4. Data Storage and Retrieval

- Data is inserted using structured queries
- Retrieval is done using SELECT operations
- Efficient indexing is applied where needed

5. Data Validation

- Input data is validated before insertion
- Required fields cannot be null
- Data types are strictly enforced

6. Data Security

- Sensitive data is protected

- SQL Injection prevention techniques are used
- Access to database is controlled

7. Data Consistency

- Database follows normalization rules
- Redundant data is minimized
- Updates are handled carefully to maintain consistency

8. Backup and Recovery

- Data backup strategies can be implemented
- Recovery methods ensure data is not lost

9. Performance Optimization

- Queries are optimized for faster execution
- Large data handled efficiently

10. Scalability

- Database design allows future expansion
- New tables and features can be added easily

Appendix D: Tools and Technologies

The system is developed using the following tools and technologies:

HTML, CSS, Bootstrap for frontend design

JavaScript for interactive features

PHP for backend processing

MySQL for database management

Leaflet.js for displaying maps

OpenStreetMap for map data